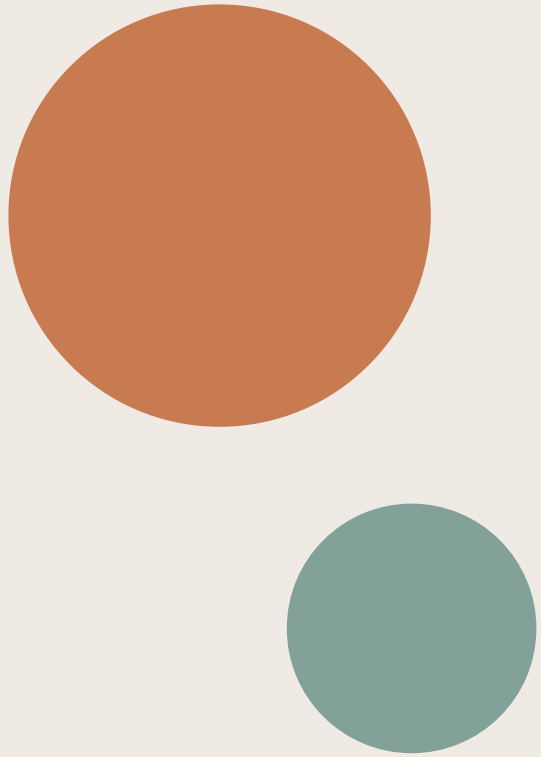
TEMA 1

Desarrollo de software y lenguajes de programación

Del código que escribimos al programa que se ejecuta.



RA1 — Software · lenguajes · traducción · JVM · herramientas

ENTORNOS DE DESARROLLO · 1.º DAM

¿Qué vamos a aprender?

Una visión general antes de entrar en la teoría.

01 Reconocer

Qué es el software, cómo se relaciona con el hardware y qué tipos encontramos.

02 Clasificar

Lenguajes de programación y diferencias entre alto y bajo nivel.

03 Explicar

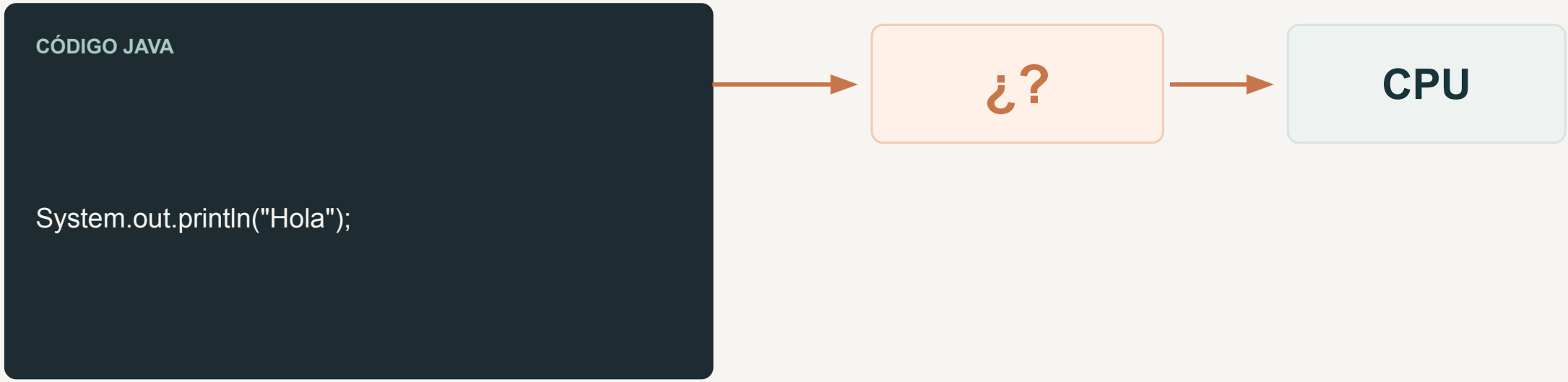
Cómo pasamos de código fuente a código ejecutable o intermedio.

04 Relacionar

Compilador, intérprete, JVM, IDE y depurador.

¿Qué ocurre cuando pulsamos “Ejecutar”?

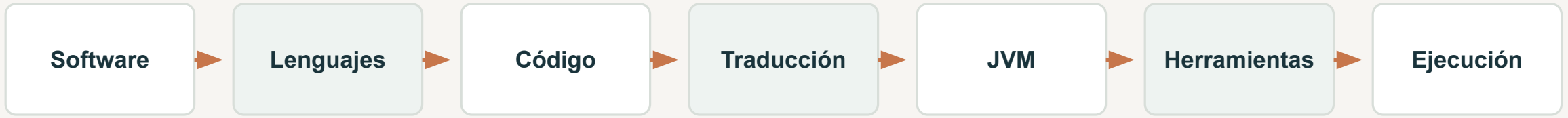
La pregunta que recorrerá todo el tema.



El procesador no lee directamente nuestro archivo .java. Entre ambos aparecen herramientas y capas que iremos descubriendo.

Ruta de aprendizaje

Cada bloque añade una pieza al proceso completo.



Meta: poder explicar con tus propias palabras qué ocurre desde que escribimos código hasta que el programa utiliza el hardware.

01

1. Software y programa informático

Comprender qué es el software, cómo se relaciona con el hardware y qué función tiene un programa.

¿Qué es un sistema informático?

Un sistema funciona gracias a elementos físicos y lógicos.



Hardware = componentes físicos · Software = programas, instrucciones y datos.

Hardware y software

Dos partes distintas que se necesitan mutuamente.

HARDWARE

Procesador, memoria RAM, almacenamiento.

Pantalla, teclado, ratón, red y periféricos.

Se puede identificar como componente físico.

SOFTWARE

Sistema operativo, aplicaciones y herramientas.

Contiene instrucciones, datos y configuraciones.

Indica qué operaciones debe realizar el sistema.

¿Cómo trabajan juntos?

El software solicita operaciones; el sistema gestiona los recursos físicos.



Ejemplo

Una app pide guardar un archivo. El sistema operativo decide cómo escribirlo en el almacenamiento.

Idea clave

El programa no controla directamente cada componente físico. Usa servicios del sistema.

Resultado

Hardware y software cooperan para realizar la tarea solicitada por el usuario.

¿Qué recursos utiliza un programa?

Un programa en ejecución consume y utiliza recursos del sistema.

01 CPU

Ejecuta instrucciones

02 RAM

Mantiene datos temporales

03 Almacenamiento

Guarda archivos y programas

04 Entrada / salida

Recibe y muestra información

05 Red

Comunica con otros equipos

¿Qué es un programa informático?

Una solución expresada como instrucciones que una máquina puede procesar.

Un programa es un conjunto organizado de instrucciones escrito para realizar una tarea o resolver un problema.

Tarea

Calcular el total de una compra.

Decisión

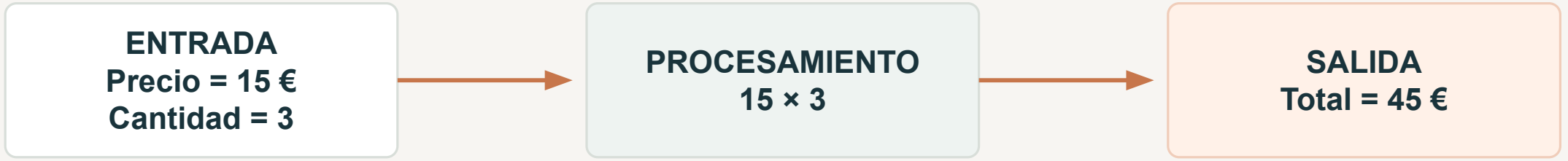
Comprobar si una nota es ≥ 5.

Gestión

Registrar alumnos, pedidos o productos.

Entrada → Procesamiento → Salida

Una forma sencilla de entender qué hace un programa.



No todos los programas siguen literalmente este esquema, pero es una buena base para comprender su comportamiento.

Programa y aplicación

En este nivel podemos utilizarlos de forma cercana, pero una aplicación real suele contener más elementos.

Programa

Conjunto de instrucciones que realiza una tarea.

Puede ser pequeño y resolver un problema muy concreto.

Aplicación

Puede reunir varios programas o módulos, bibliotecas, datos, configuración e interfaz.

Ejemplo: una aplicación de gestión académica.

Tipos de software

Una clasificación básica ayuda a entender para qué sirve cada programa.

01 Software de sistema

Gestiona el hardware y ofrece servicios básicos.

Ej.: Windows, Linux, controladores.

02 Software de aplicación

Resuelve necesidades de las personas usuarias.

Ej.: navegador, ERP, videojuego.

03 Software de desarrollo

Ayuda a crear y mantener otros programas.

Ej.: JDK, IDE, compilador.

Software de sistema

Hace posible que el equipo pueda gestionar sus recursos y ejecutar otras aplicaciones.

Sistema operativo

Administra procesos, memoria, archivos, usuarios y dispositivos.

Controladores

Permiten que el sistema operativo se comunique con dispositivos concretos.

Utilidades

Herramientas de mantenimiento, gestión o diagnóstico del sistema.

Software de aplicación

Está pensado para cubrir una necesidad concreta del usuario o de una organización.

01 Navegador

Acceder a la web

02 ERP

Gestionar procesos empresariales

03 App móvil

Ofrecer un servicio desde el teléfono

04 Videojuego

Entretenimiento interactivo

Software de desarrollo

Son herramientas que utilizaremos para construir, ejecutar y revisar otros programas.

01 Editor

Escribir código

02 Compilador

Traducir código

03 IDE

Integrar herramientas

04 Depurador

Analizar la ejecución

Actividad · ¿Qué tipo de software es?

Aplica los conceptos antes de continuar.

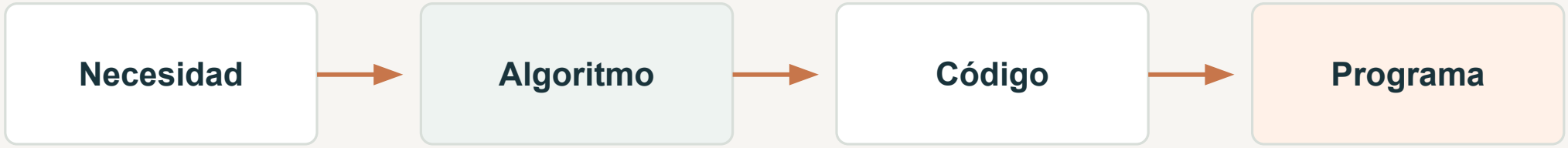
ACTIVIDAD

Clasifica: Windows, Spotify, IntelliJ IDEA, Android, Eclipse, un videojuego, un compilador y un navegador.

Justifica cada elección con una frase.

Del problema al programa

Antes de escribir código debemos saber qué problema queremos resolver.



Ejemplo

Una academia necesita registrar alumnado y notas.

Algoritmo

Secuencia ordenada de pasos para resolver una tarea antes de expresarla en un lenguaje.

02

2. Lenguajes de programación

Clasificar lenguajes y reconocer por qué no todos se utilizan para lo mismo.

¿Qué es un lenguaje de programación?

Un lenguaje formal que permite expresar instrucciones de manera precisa.

Cada lenguaje define palabras, símbolos y reglas que determinan cómo se escriben los programas.

JAVA

```
System.out.println("Hola");
```

PYTHON

```
print("Hola")
```

La intención es parecida; la sintaxis cambia.

Cada lenguaje tiene sus propias reglas

La sintaxis indica cómo debe escribirse correctamente un programa.

JAVA

```
int edad = 20;
if (edad >= 18) {
    System.out.println("Mayor");
}
```

Sintaxis

Punto y coma, llaves, palabras reservadas y estructura forman parte de las reglas del lenguaje.

Importante

Un error de sintaxis puede impedir que el código se traduzca correctamente.

¿Por qué existen tantos lenguajes?

Cada uno prioriza necesidades diferentes.

01 Rendimiento

Aprovechar recursos con eficacia

02 Facilidad

Escribir y mantener código con menos complejidad

03 Portabilidad

Ejecutar en diferentes plataformas

04 Ecosistema

Bibliotecas, herramientas y comunidad

05 Especialización

Web, sistemas, datos, móviles...

El nivel de abstracción

Cuanto más abstracto es el lenguaje, menos detalles del hardware necesitamos manejar directamente.



Más cerca del hardware

Más cerca de la forma de pensar del programador

Lenguaje máquina y bajo nivel

Trabajan muy cerca de las instrucciones y recursos del procesador.

EJEMPLO VISUAL · MÁQUINA

```
10110000 01100001
```

EJEMPLO DIDÁCTICO · ENSAMBLADOR

```
MOV AX, 5
ADD AX, 3
```

Características

- Gran control sobre el hardware.
- Menor nivel de abstracción.
- Mayor dependencia de la arquitectura.
- Requiere más conocimiento técnico.

Lenguajes de alto nivel

Permiten expresar soluciones de forma más cercana al razonamiento humano.

JAVA

```
int edad = 20;
if (edad >= 18) {
    System.out.println("Mayor de edad");
}
```

Ventajas habituales

- Mayor legibilidad.
- Desarrollo más rápido.
- Facilitan el mantenimiento.
- Mayor abstracción respecto al hardware.

Java **Python** **JavaScript** **C#**

Alto nivel vs. bajo nivel

No significa “mejor” o “peor”: responden a necesidades diferentes.

CARACTERÍSTICA	ALTO NIVEL	BAJO NIVEL
Abstracción	Alta	Baja
Relación con hardware	Indirecta	Muy cercana
Legibilidad	Generalmente mayor	Generalmente menor
Portabilidad	Suele ser mayor	Suele depender más de la arquitectura

Otras formas de clasificar lenguajes

Una misma tecnología puede describirse desde varios puntos de vista.

Según la ejecución

Compilados
Interpretados
Híbridos

Esta clasificación se desarrollará en el bloque de traductores.

Según el paradigma

Imperativos
Orientados a objetos
Funcionales

Un lenguaje puede soportar varios paradigmas.

Según el uso

Propósito general
Especializados

No todos están pensados para resolver los mismos problemas.

Lenguajes más utilizados

No hay que memorizar todos: importa reconocer características y contextos.

01 Java

Aplicaciones empresariales, multiplataforma, JVM

02 Python

Automatización, datos, IA, desarrollo rápido

03 JavaScript

Web y aplicaciones basadas en tecnologías web

04 C / C++

Sistemas, rendimiento, videojuegos, bajo nivel relativo

05 C#

Ecosistema .NET, aplicaciones y videojuegos

Java será nuestro ejemplo principal

Nos permite conectar casi todos los conceptos del RA1.

Lenguaje

Propósito general, fuertemente tipado y orientado a objetos.

Traducción

El compilador javac genera bytecode en archivos .class.

Ejecución

La JVM permite ejecutar ese bytecode en plataformas compatibles.

Python, JavaScript, C/C++ y C#

Una panorámica para reconocer usos habituales.

Python

Sintaxis compacta.
Automatización, datos, IA y web.

JavaScript

Lenguaje clave de la web.
También puede ejecutarse en servidor.

C / C++

Más cercanos a recursos y rendimiento.
Sistemas y videojuegos.

C#

Ecosistema .NET.
Aplicaciones y videojuegos.

Java ≠ JavaScript

Actividad · ¿Qué lenguaje elegirías?

Aplica los conceptos antes de continuar.

ACTIVIDAD

Relaciona varios casos con un lenguaje razonable: una web interactiva, un script de automatización, una aplicación empresarial y software de alto rendimiento.

No hay una única respuesta: justifica el criterio.

03

3. Del código fuente al programa ejecutable

Distinguir código fuente, código objeto y código ejecutable, y comprender que no son lo mismo.

Código fuente

Es el código que escribe y modifica el desarrollador.

HOLA.JAVA

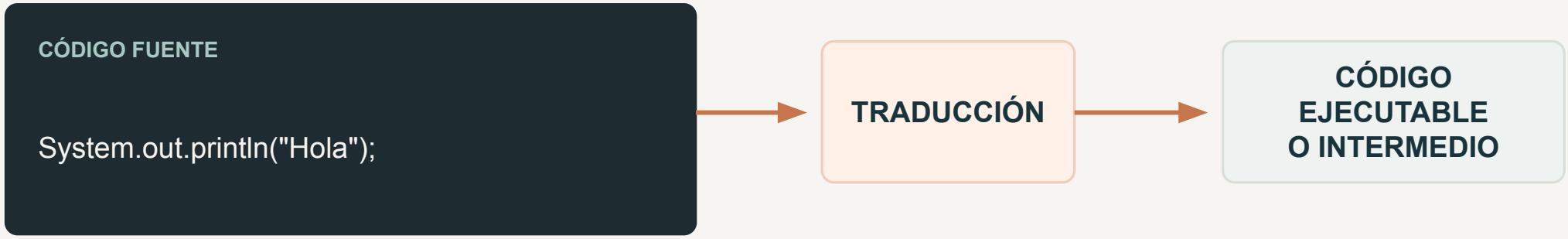
```
public class Hola {
    public static void main(String[] args) {
        System.out.println("Hola");
    }
}
```

Características

- Legible por personas que conocen el lenguaje.
- Puede modificarse.
- Se guarda en archivos como .java, .py, .c o .js.

¿Entiende la CPU nuestro código Java?

No directamente: necesitamos un proceso de traducción o ejecución intermedia.



El código fuente está pensado para desarrolladores; el sistema necesita otra representación para ejecutarlo.

Código objeto y código ejecutable

Son resultados relacionados con la traducción, pero no significan exactamente lo mismo.

Código objeto

Resultado de traducir código fuente.

Puede ser un paso intermedio que todavía deba combinarse con otros módulos o bibliotecas.

Código ejecutable

Representación preparada para iniciar un programa en un entorno concreto.

En Windows es habitual encontrar .exe, pero no todos los ejecutables usan esa extensión.

Fuente, objeto y ejecutable

Tres conceptos que el RA1 exige diferenciar.

01 Fuente

Lo escribe el programador · Legible y modificable

02 Objeto

Resultado de traducción · Puede ser intermedio

03 Ejecutable

Preparado para iniciar ejecución · Depende del entorno/plataforma

Idea clave
El proceso exacto depende del lenguaje y de la plataforma.

Actividad · Compila… pero está mal

Aplica los conceptos antes de continuar.

ACTIVIDAD

El programa calcula total = precio + cantidad cuando debería multiplicar. ¿Qué tipo de problema observas y por qué compilar no garantiza corrección?

Distingue error de sintaxis, error de ejecución y error lógico.

04

4. Traductores de lenguajes

Comprender por qué existen compiladores e intérpretes y qué diferencia hay entre ellos.

¿Por qué necesitamos un traductor?

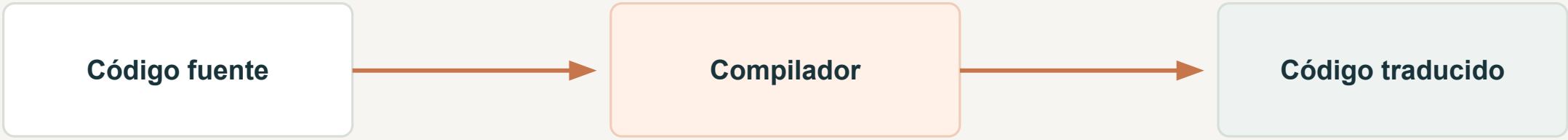
El código fuente y la representación que ejecuta la máquina no son lo mismo.



“Traductor” es una categoría general. Dos conceptos esenciales son compilador e intérprete.

El compilador

Realiza una traducción antes de la ejecución.



Qué aporta

Analiza el código y puede detectar errores sintácticos antes de generar el resultado.

Ejemplo

javac traduce archivos .java y genera bytecode .class.

El intérprete

Procesa instrucciones asociadas al momento de ejecución.



Ejemplo didáctico

Python suele utilizarse para explicar el modelo interpretado, aunque las implementaciones reales pueden realizar pasos internos adicionales.

Compilador vs. intérprete

Una comparación introductoria, no una división absoluta.

COMPILADOR

Traducción previa a la ejecución.

Puede generar código objeto, ejecutable o intermedio.

Detecta determinados errores antes de ejecutar.

INTÉRPRETE

Procesa instrucciones durante la ejecución.

No necesita generar un ejecutable independiente.

Algunos problemas aparecen al ejecutar.

Muchos sistemas modernos combinan varias técnicas.

05

5. Código intermedio y máquinas virtuales

Java nos permite ver un modelo híbrido: compilación a bytecode y ejecución sobre una máquina virtual.

El problema de las diferentes plataformas

Windows, Linux y macOS no son idénticos, ni todas las máquinas usan la misma arquitectura.

Windows

Sistema operativo y herramientas propias.

Linux

Entorno diferente, con otras combinaciones de arquitectura y software.

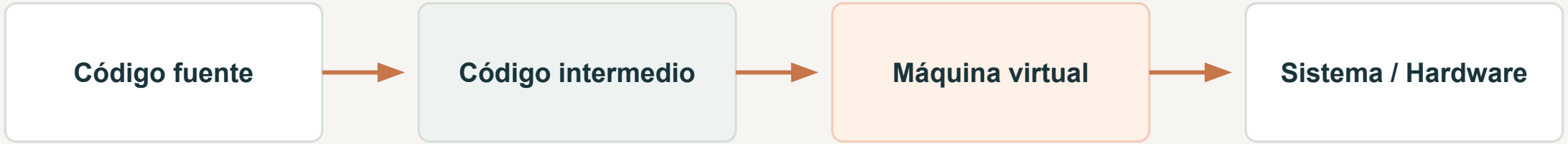
macOS

Otra plataforma sobre la que puede ejecutarse software compatible.

Pregunta: ¿podemos evitar generar una versión totalmente distinta de nuestro programa para cada plataforma?

Código intermedio y máquina virtual

Una capa adicional puede separar nuestro programa de la plataforma física.

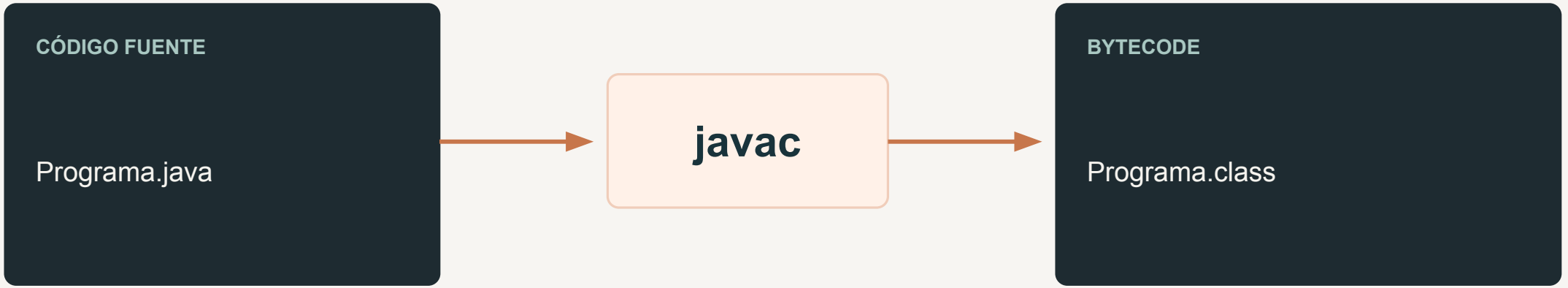


Idea clave

La máquina virtual ofrece un entorno de ejecución para ese código intermedio.

Java: del .java al .class

Primero compilamos el código fuente con javac.



El archivo .class no contiene el código fuente original: contiene bytecode.

Bytecode y JVM

La JVM verifica y ejecuta el bytecode Java.

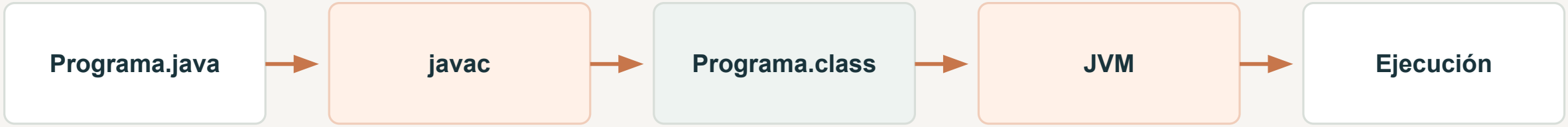


Importante

La JVM es software: actúa como intermediaria entre el bytecode y la plataforma donde se está ejecutando.

El proceso completo de Java

Una de las diapositivas clave del tema.



Portabilidad: el mismo bytecode puede ejecutarse en sistemas diferentes si disponen de una JVM compatible.

JVM Windows

JVM Linux

JVM macOS

JVM, JRE y JDK

Tres siglas frecuentes en Java que conviene distinguir.

JVM

Java Virtual Machine.

Ejecuta el bytecode Java y actúa como capa de ejecución.

JRE

Entorno de ejecución de Java en la terminología tradicional.

Incluye lo necesario para ejecutar aplicaciones.

JDK

Java Development Kit.

Incluye herramientas para desarrollar, como el compilador javac.

06

6. Herramientas de programación

Reconocer qué aporta cada herramienta sin entrar todavía en la configuración profunda de un IDE.

Editor e IDE

Un IDE integra varias herramientas que por separado también existen.

EDITOR DE CÓDIGO

Facilita escribir y modificar código.

Resaltado de sintaxis, numeración de líneas, búsqueda, autocompletado y navegación.

IDE

Integrated Development Environment.

Integra editor, ejecución, compilación, depuración y gestión del proyecto en una interfaz.

Cada herramienta tiene una función

El alumnado debe poder reconocerlas y explicar para qué sirven.

01 Editor

Escribir y modificar código

02 Compilador

Traducir código

03 Intérprete

Procesar instrucciones en ejecución

04 JVM

Ejecutar bytecode Java

05 IDE

Integrar herramientas

06 Depurador

Analizar la ejecución paso a paso

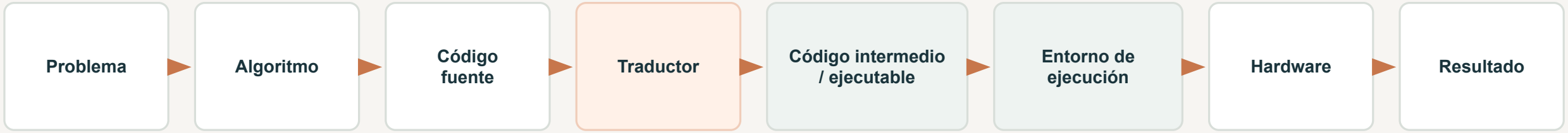
07

7. Del código fuente al programa en funcionamiento

Integrar todo el tema en un único recorrido desde la necesidad inicial hasta el hardware.

Todo el proceso

Ahora podemos responder a la pregunta inicial del tema.



Lectura del recorrido

Partimos de una necesidad, la expresamos como código, la traducimos y finalmente el programa se ejecuta utilizando recursos del sistema.

En Java, el recorrido central es: .java → javac → .class → JVM → ejecución.

Actividad final · Ordena el proceso

Aplica los conceptos antes de continuar.

ACTIVIDAD

Ordena y relaciona estos elementos: editor, Programa.java, javac, Programa.class, JVM, sistema operativo, CPU y resultado.

Después explica qué hace cada elemento en una frase.

Resumen del Tema 1

Cinco ideas que debes poder explicar al terminar.

1. Software

Hardware y software cooperan; un programa utiliza recursos del sistema.

2. Lenguajes

Existen distintos niveles, paradigmas y formas de ejecución.

3. Tipos de código

Fuente, objeto, ejecutable e intermedio no significan lo mismo.

4. Traducción

Compiladores e intérpretes conectan el código con su ejecución.

5. Java y herramientas

javac → bytecode → JVM; el IDE integra herramientas que facilitan el proceso.

Pregunta final: ¿qué ocurre realmente cuando pulsas “Ejecutar”?